# Automatic Generation of Formulae for Decidable Fragments of First-Order Logic

Manuele Borgia

Anno Accademico 2025-2026

# Contents

# Introduction

The automation of logical reasoning has been a central challenge in computer science since its earliest days. First-Order Logic (FOL), formalized in the foundations established by Frege's *Begriffsschrift* and later developed by Russell and Whitehead in *Principia Mathematica* [16], provides a language expressive enough to capture mathematical reasoning, while maintaining the structural precision required to admit computational treatment. Its applications span knowledge representation, formal verification, and artificial intelligence, where the ability to reason automatically over logical formulae has practical consequences ranging from software correctness to ontology reasoning.

While highly expressive, FOL is fundamentally constrained by the undecidability of its *satisfiability problem*, a theoretical limit proven by Church and Turing in the 1930s. This limitation has motivated two complementary lines of research. The first is the development of Automated Theorem Provers (ATPs) — systems such as Vampire [10], which employ saturation-based calculi and sophisticated heuristics to handle large classes of problems efficiently, operating within the theoretical bounds of semi-decidability. The second is the identification of *decidable fragments* of FOL: syntactically restricted subsets that retain sufficient expressive power for practical applications while admitting algorithms that always terminate with a correct answer.

This research analyzes four of the most studied decidable fragments: the Two-Variable Fragment ($\mathrm{FO}^2$), the Guarded Fragment (GF), the Fluted Fragment (FF), and the Unary Negation Fragment (UNF). Each imposes distinct syntactic constraints — on the number of variables, the structure of quantification, or the use of negation — and each captures different classes of reasoning problems that arise naturally in areas such as description logics, database theory, and computational linguistics.

The practical evaluation of decision procedures for these fragments requires extensive testing on formulae that faithfully represent the syntactic diversity of each class. This need motivates the central contribution of this thesis: the design and implementation of **parametric random formula generators** for decidable fragments of FOL. Rather than relying on existing benchmark libraries, which are often sparse or biased toward specific problem structures, our generators produce formulae that are syntactically correct by construction, with configurable structural parameters controlling depth, predicate vocabulary, and quantifier density. A probabilistic *budget model* governs the generation pro-

cess, enabling systematic control over formula complexity. Generated formulae are validated using Vampire, ensuring that satisfiability labels are reliable. For the Two-Variable Fragment, a dedicated finite model construction provides an alternative, model-theoretic approach to generating satisfiable instances.

This thesis is organized as follows. Chapter 1 introduces the theoretical foundations of First-Order Logic. Chapter 2 presents the decidable fragments addressed by this work. Chapter 4 details the implementation and the probabilistic budget model of the formula generator. Finally, Chapter **??** discusses the validation process using Vampire.

# Chapter 1

# First-Order Logic

This chapter establishes the theoretical foundations upon which this thesis is built. We present a concise yet rigorous overview of First-Order Logic (FOL), detailing its syntax and semantics, and concluding with the fundamental undecidability of its satisfiability problem. This theoretical limit motivates the exploration of decidable fragments, which constitutes the core focus of this work.

## 1.1 Syntax

The syntax of First-Order Logic strictly defines the rules for constructing valid mathematical expressions. It is built upon a specific alphabet known as a signature [4].

A **signature** (or vocabulary) $\sigma$ consists of a set of predicate symbols, each associated with a fixed *arity* $n \geq 0$, a set of function symbols, each with a fixed arity $n \geq 0$ (where arity-0 function symbols are conventionally called *constants*), and a countably infinite set of *variables* $\mathcal{V} = \{x_1, x_2, x_3, \ldots\}$.

**Definition 1.1** (Terms)**.** *The set of* terms *over a signature $\sigma$ is defined inductively as follows:*

- *every variable $x \in \mathcal{V}$ is a term;*
- *if $f$ is a function symbol of arity $n$ and $t_1, \ldots, t_n$ are terms, then $f(t_1, \ldots, t_n)$ is a term.*

*A term containing no variables is referred to as a* ground term*.*

**Definition 1.2** (Formulae)**.** *The set of* first-order formulae *over $\sigma$ is defined inductively:*

- ***Atomic formulae.*** *If $P$ is a predicate symbol of arity $n$ and $t_1, \ldots, t_n$ are terms, then $P(t_1, \ldots, t_n)$ is an* atom*. Additionally, the expression $t_1 = t_2$ is an* equality atom*.*

- ***Boolean connectives.*** *If $\varphi$ and $\psi$ are formulae, then so are $\neg\varphi$, $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, and $(\varphi \to \psi)$.*
- ***Quantifiers.*** *If $\varphi$ is a formula and $x \in \mathcal{V}$, then $\exists x\, \varphi$ and $\forall x\, \varphi$ are formulae.*

An occurrence of a variable $x$ in a formula $\varphi$ is considered *bound* if it lies within the scope of a quantifier ($\exists x$ or $\forall x$); otherwise, it is *free*. A formula containing no free variables is called a *sentence*.

**Definition 1.3** (Negation Normal Form)**.** *A formula is in* Negation Normal Form *(NNF) if the negation operator ($\neg$) is applied exclusively to atomic formulae. Every FOL formula can be transformed into a logically equivalent formula in NNF by systematically pushing negations inward. This is achieved by applying De Morgan's laws and the duality of quantifiers ($\neg\forall x\, \varphi \equiv \exists x\, \neg\varphi$ and $\neg\exists x\, \varphi \equiv \forall x\, \neg\varphi$).*

### 1.1.1 Formulae as Parse Trees

While formulae are typically written as linear strings of text, their inductive definition implies a strict hierarchical structure. In formal logic and computer science, a well-formed formula can be unambiguously represented as a *parse tree* (or abstract syntax tree) [9]. In this tree representation, the internal nodes correspond to logical connectives and quantifiers, while the leaf nodes represent the atomic formulae.

For example, consider a simple academic domain where $S(x)$ denotes "$x$ is a student", $C(x)$ denotes "$x$ is a course", and $E(x, y)$ denotes "$x$ is enrolled in $y$". The statement *"Every student is enrolled in at least one course"* can be formalized as $\forall x(S(x) \to \exists y(C(y) \wedge E(x, y)))$. The tree structure underlying this mathematical statement is illustrated in Figure 1.1.

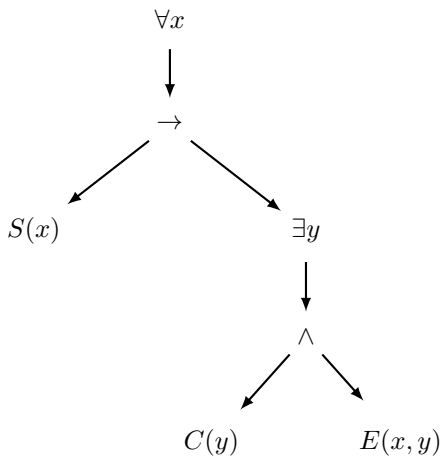


Figure 1.1: The hierarchical parse tree representing the syntactic structure of the formula $\forall x(S(x) \rightarrow \exists y(C(y) \wedge E(x, y)))$.

This hierarchical view is mathematically crucial because it dictates the precise order of operations during semantic evaluation and structural transformations.

## 1.2 Semantics

While syntax dictates the grammatical structure of formulae, semantics provides their mathematical meaning by evaluating them against a specific interpretation [5].

**Definition 1.4** (Structure)**.** *A* $\sigma$-structure *(or model)* $\mathcal{M}$ *consists of:*

- *a non-empty set* $\Delta^{\mathcal{M}}$*, called the* domain *(or* universe*);*
- *for each predicate symbol* $P$ *of arity* $n$*, a relation* $P^{\mathcal{M}} \subseteq (\Delta^{\mathcal{M}})^n$*;*
- *for each function symbol* $f$ *of arity* $n$*, a function* $f^{\mathcal{M}} : (\Delta^{\mathcal{M}})^n \rightarrow \Delta^{\mathcal{M}}$*.*

**Definition 1.5** (Interpretation and Satisfaction)**.** *To evaluate formulae containing free variables, we introduce a* variable assignment $\beta : \mathcal{V} \rightarrow \Delta^{\mathcal{M}}$*, which maps variables to specific domain elements. The* satisfaction relation*, denoted as* $\mathcal{M}, \beta \models \varphi$*, is defined inductively:*

- $\mathcal{M}, \beta \models P(t_1, \ldots, t_n)$ *iff* $(t_1^{\mathcal{M}}[\beta], \ldots, t_n^{\mathcal{M}}[\beta]) \in P^{\mathcal{M}}$*;*
- $\mathcal{M}, \beta \models t_1 = t_2$ *iff* $t_1^{\mathcal{M}}[\beta] = t_2^{\mathcal{M}}[\beta]$*;*
- $\mathcal{M}, \beta \models \neg\varphi$ *iff* $\mathcal{M}, \beta \not\models \varphi$*;*

- $\mathcal{M}, \beta \models \varphi \wedge \psi$ *iff* $\mathcal{M}, \beta \models \varphi$ *and* $\mathcal{M}, \beta \models \psi$*;*
- $\mathcal{M}, \beta \models \exists x\, \varphi$ *iff there exists at least one element* $d \in \Delta^{\mathcal{M}}$ *such that* $\mathcal{M}, \beta[x \mapsto d] \models \varphi$*;*
- $\mathcal{M}, \beta \models \forall x\, \varphi$ *iff for all elements* $d \in \Delta^{\mathcal{M}}$*, it holds that* $\mathcal{M}, \beta[x \mapsto d] \models \varphi$*.*

*For a sentence* $\varphi$ *(which has no free variables, making the assignment* $\beta$ *irrelevant), we simply write* $\mathcal{M} \models \varphi$ *and state that* $\mathcal{M}$ *is a* model *of* $\varphi$*.*

### 1.2.1 Concrete Evaluation over a Finite Domain

To illustrate the semantic evaluation process practically, consider a purely relational signature with a unary predicate $P$ and a binary relation $R$. We can define a finite structure $\mathcal{M}$ with a domain $\Delta^{\mathcal{M}} = \{1, 2, 3\}$. Let the interpretations be:

- $P^{\mathcal{M}} = \{1, 2\}$
- $R^{\mathcal{M}} = \{(1, 2), (2, 3)\}$

Consider the sentence $\exists x(P(x) \wedge \exists y\, R(x, y))$. To determine if $\mathcal{M}$ is a model for this sentence, we evaluate the truth condition: is there an element in $\{1, 2, 3\}$ that satisfies both $P$ and has an outgoing $R$-relation? If we assign $x \mapsto 2$, we see that $2 \in P^{\mathcal{M}}$. Furthermore, assigning $y \mapsto 3$ satisfies the relation $(2, 3) \in R^{\mathcal{M}}$. Because we have found a valid assignment, the sentence is true in this structure ($\mathcal{M} \models \varphi$).

**Definition 1.6** (Satisfiability and Validity)**.** *A first-order formula* $\varphi$ *is classified as:*

- satisfiable *if there exists at least one structure* $\mathcal{M}$ *and assignment* $\beta$ *such that* $\mathcal{M}, \beta \models \varphi$*;*
- valid *(or a* tautology*) if* $\mathcal{M}, \beta \models \varphi$ *holds for every possible structure* $\mathcal{M}$ *and every assignment* $\beta$*;*
- unsatisfiable *if it has no model.*

*It immediately follows that a formula* $\varphi$ *is valid if and only if its negation* $\neg\varphi$ *is unsatisfiable.*

## 1.3 Decidability and Undecidability

The expressiveness of First-Order Logic makes it an extremely powerful tool for formalizing mathematics and computer science. However, this expressive power comes at a severe computational cost.

### 1.3.1 Semi-Decidability

Before discussing the absolute limits of FOL, it is crucial to recognize Gödel's Completeness Theorem (1930) [6], which establishes that FOL is *semi-decidable* (or recursively enumerable). This theorem guarantees that if a formula $\varphi$ is truly valid, there exists a finite formal proof for it. Consequently, a systematic algorithm can eventually find this proof and confirm its validity in a finite amount of time.

However, the semi-decidability property provides no guarantees for formulae that are *not* valid. If an automated reasoning system attempts to prove an invalid formula, it may search for a proof indefinitely, cycling through infinite domains without ever halting to report a negative result.

**Theorem 1.1** (Undecidability of FOL [3, 14])**.** *The satisfiability problem for full first-order logic is undecidable: there exists no general algorithm that, given an arbitrary FOL sentence, can reliably determine in a finite amount of time whether the sentence is satisfiable.*

This profound result, established independently by Alonzo Church and Alan Turing in the 1930s, places a fundamental theoretical limit on automated reasoning over unrestricted FOL.

This inherent undecidability has directly motivated the systematic study of syntactically restricted *fragments* of FOL. By limiting certain syntactic features—such as the number of available variables, the arity of predicates, or the allowed quantifier prefixes—researchers have identified subclasses of logic whose satisfiability problem *is* fully decidable. The algorithmic generation and semantic testing of these specific decidable fragments lie at the very heart of the research and software developed for this thesis.

# Chapter 2

# Decidable Fragments

The fundamental undecidability of First-Order Logic necessitates the study of syntactically restricted *fragments* that achieve decidability while retaining enough expressivity for practical applications. This chapter introduces the decidable fragments targeted by our thesis.

**Note on function symbols.** The decidable fragments studied in this thesis are defined over a *purely relational* signature, i.e., a signature containing only predicate symbols, variables, and possibly constants. Function symbols of arity $\geq 1$ are excluded, as they lead to undecidability. All logical analyses described in this thesis therefore operate within purely relational signatures.

## 2.1 The Two-Variable Fragment

The two-variable fragment of First-Order Logic, denoted by $FO^2$, is defined as the set of all FOL formulae that use at most two distinct variable names (typically $x$ and $y$). Introduced in the context of the decision problem, $FO^2$ is one of the most prominent decidable fragments, balancing expressive power with a complexity class that remains within NExpTime [8].

The syntax of $FO^2$ is defined inductively. Let $\tau$ be a relational signature and $V = \{x, y\}$ be the set of available variables. The set of $FO^2$ formulae is defined as the smallest set such that:

**Definition 2.1** (Two-Variable Formula)**.**

1. ***Atomic:***
   *Every atomic formula $P(t_1, \ldots, t_n)$ using variables strictly from $V$ is an $FO^2$ formula. Additionally, if $v_1, v_2 \in V$, then the equality $v_1 = v_2$ is an $FO^2$ formula.*

2. ***Boolean:***
   *If $\phi$ and $\psi$ are $FO^2$ formulae, then $\neg\phi$, $\phi \wedge \psi$, and $\phi \vee \psi$ are $FO^2$ formulae.*

3. ***Quantification:***
   *If $\phi$ is an $FO^2$ formula and $z \in \{x, y\}$, then $\exists z\phi$ and $\forall z\phi$ are $FO^2$ formulae.*

### 2.1.1 Tree Representation and Variable Recycling

From a generative perspective, $FO^2$ formulae, like those in other fragments, are constructed via a top-down traversal of an Abstract Syntax Tree (AST). However, the generative constraint here differs significantly from the Fluted or Guarded fragments. Instead of maintaining a growing stack of variables (as in FF) or ensuring local variable coverage via guard atoms (as in GF), the AST traversal for $FO^2$ is globally bounded by the fixed variable pool $V = \{x, y\}$.

This imposes a *variable recycling* mechanism during the tree traversal. When constructing a deep AST branch, quantifiers must continuously reuse $x$ and $y$, effectively "shadowing" or overwriting the variable bindings from higher up in the tree. For example, a valid $FO^2$ nested structure might look like the formula in Equation (2.1):

$$\forall x\Big(P(x) \vee \exists y\big(R(x,y) \wedge \exists x(S(y,x))\big)\Big) \tag{2.1}$$

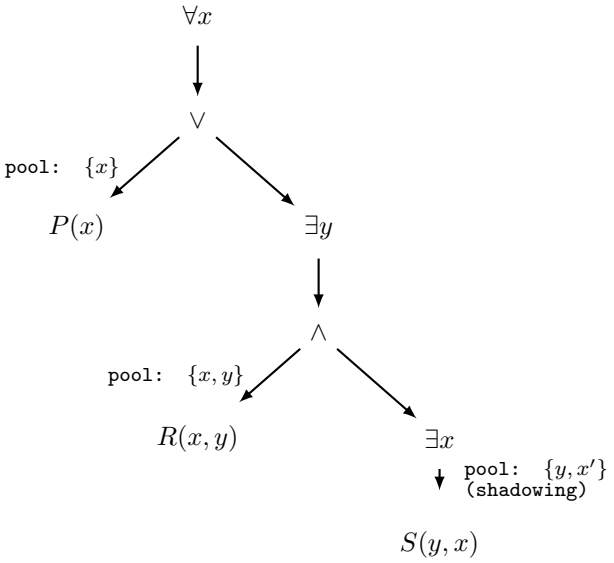


Figure 2.1: Syntactic tree of the $FO^2$ formula (2.1). The active variable pool is passed down the tree and strictly bounded to $\{x, y\}$. Notice how the innermost existential quantifier over $x$ rebinds the variable (denoted as $x'$), shadowing the outermost universal binding.

As illustrated in Figure 2.1, the innermost existential quantifier over $x$ rebinds the variable, severing the connection to the outermost universal quantifier. For the generator, this means that tracking the available variables is trivial (it is always a choice between $x$ and $y$), but ensuring that the generated AST pro-

duces meaningful, non-trivial satisfiability challenges requires careful handling of this rebinding mechanism.

### 2.1.2 Finite Model Property and SAT Generation

A defining characteristic of $FO^2$ is that it possesses the *finite model property*. Originally established by Mortimer [**?**], the property states that if an $FO^2$ formula is satisfiable, it is satisfiable in a finite model. Later, Grädel, Kolaitis, and Vardi [8] sharpened this result, proving that any satisfiable $FO^2$ formula has a model whose size is bounded exponentially by the size of the formula (specifically, a domain of size at most $2^{O(|\phi|)}$).

This theoretical foundation provides a powerful mechanism for procedural SAT generation. Because we are guaranteed that a finite model suffices, we can generate satisfiable $FO^2$ instances by constructively evaluating candidate abstract syntax trees against a pre-defined finite model. By fixing a finite domain (a bounded *domain size*) and randomly generating interpretations for the relational vocabulary, a generator can traverse the AST top-down, propagating a set of target variable assignments (i.e., tuples $\langle x, y \rangle$).

At each generative step, an evaluator evaluates the partially constructed formula under the current assignments over the finite model. Subtrees are generated such that they evaluate to true for the specified target configurations, enforcing satisfiability by construction without requiring an external SMT solver to find a model *a posteriori*.

### 2.1.3 Complexity

The satisfiability problem for $FO^2$ is known to be **NExpTime-complete** [8]. The high complexity class reflects the expressive power of the fragment, which, despite the two-variable restriction, can still concisely encode exponentially large grids and complex relational structures through variable recycling.

From the perspective of procedural generation, this complexity translates into combinatorial challenges during the target-driven SAT construction. Because universal quantifiers ($\forall$) and negation ($\neg$) require evaluating and complementing variable assignments across the entire finite domain, the number of target assignments can grow rapidly. For instance, complementing a target set requires computing the difference between the full domain cross-product and the current targets.

To mitigate the computational overhead associated with the NExpTime nature of the fragment, practical implementations must employ bound limits (e.g., capping the maximum number of targets propagated) and utilize sparse data structures. This ensures that the SAT construction remains computationally feasible while still producing logically deep and mathematically valid $FO^2$ formulae.

## 2.2 The Fluted Fragment

The landscape of decidable fragments is characterized by the trade-off between expressivity and complexity. By contrast to fragments defined by variable counting (like $FO^2$), the Fluted Fragment (FF) imposes a restriction on the ordering of variables. This variable-ordering discipline is significant because it preserves predicate arity and formula structure, focusing entirely on the logical application of predicates to variable sequences.

FF acts as a generalization of modal logic, sharing characteristics with the guarded fragments; however, it allows the negation of relation atoms, enabling the capture of enriched modal logics. Originally stemming from the work of Quine [12], the fragment's decidability is modernized and formalized by Pratt-Hartmann, Szwast, and Tendera [11].

The fundamental constraint of FF is the *suffix property*. An atomic formula is fluted if its arguments form a contiguous suffix of the variables currently in scope.

**Definition 2.2** (Fluted Formula)**.** *The set of fluted formulae over $X_i$ (where $0 \leq i \leq m$) is defined inductively:*

1. ***Atomic:***
   *For any $n$-ary predicate $P \in \mathcal{P}$ such that $n \leq i$, the atomic formula $P(x_{i-n+1}, \ldots, x_i)$ is fluted over $X_i$.*

2. ***Boolean:***
   *If $\phi$ and $\psi$ are fluted over $X_i$, then $\neg\phi$, $\phi \wedge \psi$, $\phi \vee \psi$, and $\phi \Rightarrow \psi$ are fluted over $X_i$.*

3. ***Quantification:***
   *If $\phi$ is fluted over $X_{i+1}$, then $\exists x_{i+1}\phi$ and $\forall x_{i+1}\phi$ are fluted over $X_i$.*

This inductive structure entails that the quantification step expands the active variable scope, while atomic leaves are strictly bounded to the most recently introduced variables.

### 2.2.1 Tree Representation and the Scope Stack

From a generative perspective, the structure of a fluted formula is best understood through its Abstract Syntax Tree (AST). The variable-ordering discipline of FF naturally translates to a stack-based traversal during top-down generation.

Unlike $FO^2$, which recycles a fixed pool of variables, building an AST for the Fluted Fragment requires maintaining a dynamic "scope stack" of quantified variables that is passed down to child nodes. Each quantifier node conceptually pushes a new variable onto this stack.

Consider the following valid fluted formula:

$$\forall x_1 \forall x_2 \Big( P(x_1, x_2) \wedge \exists x_3 \big( R(x_2, x_3) \big) \Big) \tag{2.2}$$

Its syntactic structure can be visually represented as an AST, as shown in Figure 2.2.

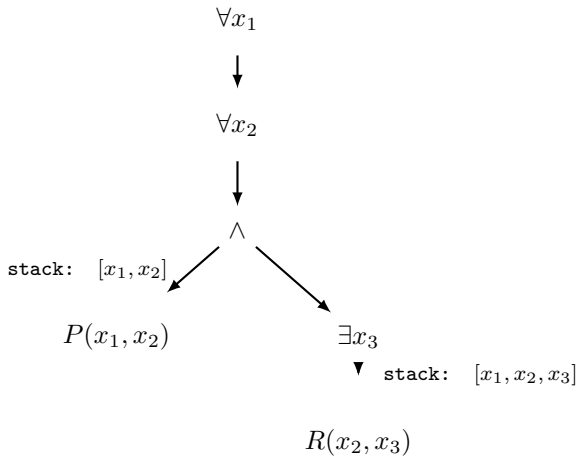


Figure 2.2: Syntactic tree of the fluted formula (2.2). The active variable stack is passed down the tree, and atomic leaves deterministically consume a contiguous suffix of it.

When the AST traversal reaches a leaf node (an atomic formula), the suffix property acts as a strict generative constraint. For instance, when generating the atom $R$ of arity 2 in Figure 2.2, the active stack is $[x_1, x_2, x_3]$. To comply with FF rules, the atom must exclusively consume the contiguous suffix $(x_2, x_3)$. Attempting to construct an atom such as $Q(x_1, x_3)$ at that exact position would invalidate the formula. This inherent top-down predictability allows for an efficient, stack-driven generation strategy.

### 2.2.2 Complexity

The general Fluted Fragment exhibits non-elementary complexity, which scales heavily with the number of variables $m$ [11]. This theoretical landscape poses a severe computational challenge, as attempting to prove unsatisfiability in FF can require exploring an enormously vast state space. In the context of this thesis, this justifies the necessity of our generation strategy: by algorithmically constructing structured FF formulae, we provide theorem provers with strictly compliant input to benchmark their limits against this non-elementary complexity.

## 2.3 The Guarded Fragment

The Guarded Fragment (GF), introduced by Andréka, van Benthem, and Németi [1], imposes structural constraints on the interaction between quantifiers and their sub-formulae. Quantification must be "guarded" by atomic formulae, which effectively restricts the domain of the quantifier to tuples satisfying a specific relational constraint.

The syntax of GF is defined inductively. Let $\tau$ be a relational signature. The set of guarded formulae is defined as the smallest set such that:

**Definition 2.3** (Guarded Formula)**.**

1. ***Atomic:***
   *Every atomic formula $P(t_1, \ldots, t_n)$ is a guarded formula.*
2. ***Boolean:***
   *If $\phi$ and $\psi$ are guarded formulae, then $\neg\phi$ and $\phi \wedge \psi$ are guarded formulae.*
3. ***Quantification:***
   *Let $\phi(\bar{x}, \bar{y})$ be guarded and $G(\bar{x}, \bar{y})$ be an atomic guard containing all free variables of $\phi$. Then $\exists\bar{y}(G(\bar{x}, \bar{y}) \wedge \phi)$ and $\forall\bar{y}(G(\bar{x}, \bar{y}) \Rightarrow \phi)$ are guarded formulae.*

GF satisfies the *tree model property* [1], meaning every satisfiable guarded formula can be satisfied in a tree-structured model. This is the cornerstone of its decidability [7].

### 2.3.1 Tree Representation and Guard Generation

From a generative perspective, constructing a guarded formula requires a significantly different Abstract Syntax Tree (AST) traversal compared to the Fluted Fragment. While FF maintains a linear stack to enforce the suffix property, the generation of GF formulae relies on tracking a dynamic set of currently active free variables. In our top-down AST generation, whenever a quantifier over a set of new variables $\bar{y}$ is introduced, the fragment's rules dictate that a guard atom $G(\bar{x}, \bar{y})$ must be explicitly constructed, where $\bar{x}$ represents the variables currently in the active scope. To satisfy this, the generator must query the available vocabulary for a predicate whose arity exactly matches the combined size of the active scope and the newly bound variables.

Consequently, a quantified node in the AST is not a simple unary wrapper but a composite structure. For an existential quantification, the AST node branches into an $\wedge$ connective (or an $\rightarrow$ connective for universal quantification), with the specifically tailored guard atom $G$ as its left child and the recursive sub-formula $\phi$ as its right child. The traversal then proceeds to generate the body of $\phi$ with the expanded active scope $\bar{x} \cup \bar{y}$. If no predicate of the required arity is available in the vocabulary to form a valid guard, the branch is deemed invalid, and the generator must backtrack by triggering a retry exception.

An elegant example of this generative structure is shown in Equation (2.3):

$$\exists x_2\Big(G(x_1, x_2) \wedge \forall x_3\big(H(x_1, x_2, x_3) \rightarrow (P(x_2) \vee S(x_1, x_3))\big)\Big) \tag{2.3}$$

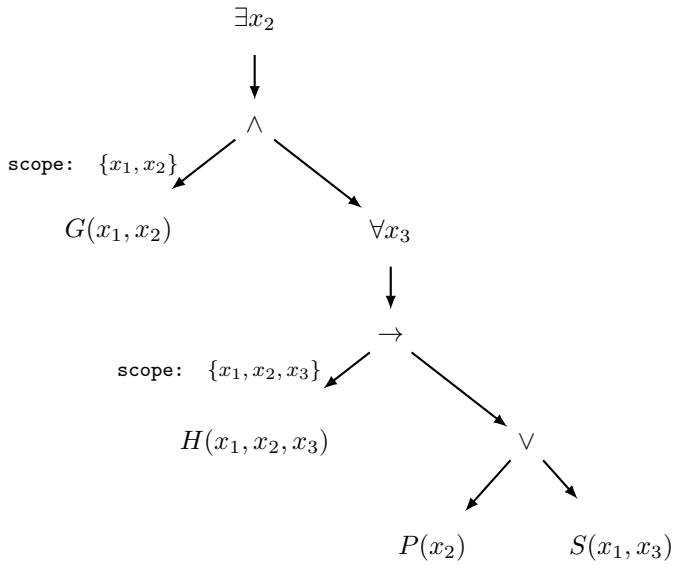


Figure 2.3: Syntactic tree of the guarded formula (2.3) assuming an initial active variable $x_1$. The dynamic active scope grows as quantifiers are introduced. Each quantification spawns a composite node structure where an explicit guard atom shields the sub-formula, strictly consuming the current variable scope.

As illustrated in Figure 2.3, this explicit structural constraint ensures that every bound variable is immediately shielded by a guard atom ($G$ and $H$, respectively) that spans the entire active scope. The magnificence of this generative approach lies in its structural determinism: the complex syntactic restrictions of the Guarded Fragment are not enforced *post-hoc* through rejection sampling, but are structurally guaranteed *by design* during the AST traversal. The tree is forced to interleave logical connectives with specific atoms from the vocabulary whose arities perfectly match the dynamically expanding scope, thus translating theoretical logical boundaries into an elegant and highly constrained algorithmic generation pipeline.

### 2.3.2 Complexity

The satisfiability problem for the Guarded Fragment is ExpTime-complete [7]. This result highlights a significant increase in expressive power compared to basic propositional modal logic (which is typically PSpace-complete), while remaining much more tractable than the non-elementary complexity of the Fluted Fragment. Benchmarking provers against GF instances is particularly valuable for observing how solvers handle deeply nested local constraints without being overwhelmed by global variable dependencies.

## 2.4 The Unary Negation Fragment

The Unary Negation Fragment (UNFO), introduced by ten Cate and Segoufin [13], restricts the scope of negation rather than the quantifier pattern. While the Guarded Fragment restricts the placement of quantifiers to achieve a tree-like model, UNFO allows arbitrary quantification but isolates the primary source of undecidability in FOL: the unrestricted use of negation over multiple variables.

The syntax of UNFO is defined inductively. Let $\tau$ be a relational signature. The set of UNFO formulae is defined as the smallest set such that:

**Definition 2.4** (Unary Negation Formula)**.**

1. ***Atomic:***
   *Every atomic formula is an UNFO formula.*
2. ***Boolean:***
   *If $\phi$ and $\psi$ are UNFO formulae, then $\phi \wedge \psi$ and $\phi \vee \psi$ are UNFO formulae.*
3. ***Negation:***
   *If $\phi$ is an UNFO formula such that $|free(\phi)| \leq 1$, then $\neg\phi$ is an UNFO formula.*
4. ***Quantification:***
   *If $\phi$ is an UNFO formula, then $\exists x\phi$ and $\forall x\phi$ are UNFO formulae.*

Decidability in UNFO is maintained by confining the complexity of negation to formulae with at most one free variable, effectively preventing the construction of undecidable contradictions in higher-arity predicates [13].

### 2.4.1 Tree Representation and Variable Dependencies

From a structural perspective, the syntax of an UNFO formula can be visualized through an Abstract Syntax Tree (AST), where the constraints of the fragment dictate the flow of variable dependencies. Unlike the Fluted Fragment (which relies on a strict variable stack) or the Guarded Fragment (which forces guard atoms at every quantification step), UNFO is highly permissive with positive connectives and quantifiers. The structural "bottleneck" occurs exclusively at the negation nodes.

Consider the following valid UNFO formula:

$$\forall x\Big(P(x) \vee \exists y\big(R(x,y) \wedge \neg Q(y)\big)\Big) \tag{2.4}$$

In this formula, the negation is applied solely to the sub-formula $Q(y)$, which contains exactly one free variable ($y$). This satisfies the unary restriction. Its AST representation is shown in Figure 2.4.

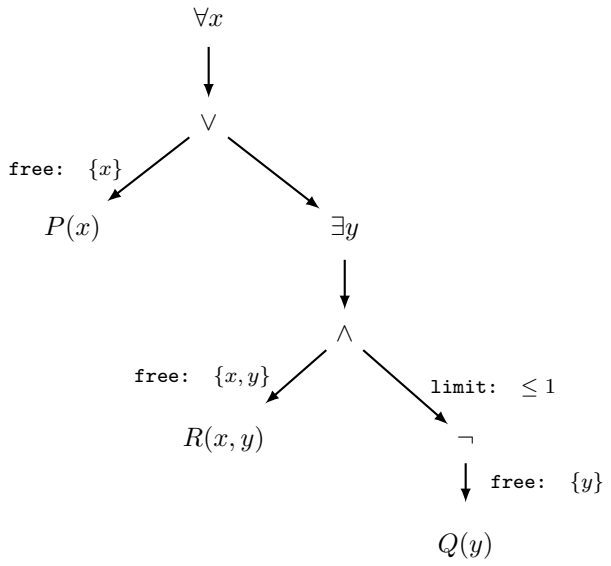


Figure 2.4: Syntactic tree of the valid UNFO formula (2.4). The negation node acts as a strict structural filter, ensuring that the sub-tree below it depends on at most one free variable.

As illustrated in Figure 2.4, the positive branches (like the one leading to $R(x, y)$) can freely accumulate multiple variables in their scope. However, when the AST branches into a negation node, the set of free variables must be aggressively restricted.

Conversely, consider the following invalid formula:

$$\forall x \exists y\big(\neg R(x,y)\big) \tag{2.5}$$

This construction strictly violates the UNFO rules because the sub-formula under the negation, $R(x, y)$, contains two free variables ($x$ and $y$). This creates a structural topology where branches underneath a negation node are inherently isolated in terms of variable dependencies. This architectural behavior perfectly mirrors the theoretical foundation of UNFO: it allows complex positive relations but forbids non-local negated structures, neutralizing the main trigger of undecidability in unrestricted FOL.

### 2.4.2 Complexity

The complexity of the satisfiability problem for UNFO is 2ExpTime-complete in the general case, but it drops to ExpTime-complete if the maximum arity of the predicates in the signature is fixed [13]. This makes UNFO an incredibly powerful theoretical tool: it offers a much larger expressive footprint than GF—allowing properties over arbitrarily many variables without guard atoms—while sitting within a comparable, well-understood complexity class.

## 2.5 Propositional Modal Logic and the Standard Translation

While the fragments discussed thus far ($FO^2$, FF, GF, UNFO) are defined purely within the vocabulary of First-Order Logic, their origins, properties, and constraints are deeply intertwined with Propositional Modal Logic (ML). Historically, modal logic was developed to reason about necessity and possibility, but from a modern computational perspective, it is best understood as a robustly decidable, tree-like logic for reasoning about relational structures, often evaluated over Kripke semantics.

The fundamental insight that connects ML to the decidable fragments of FOL is that modal operators can be viewed as "macros" for specific, heavily restricted patterns of first-order quantification. This connection establishes modal logic as the baseline or "minimal" decidable fragment, out of which fragments like GF and UNFO evolved as expressive generalizations.

### 2.5.1 The Standard Translation

The formal bridge between Propositional Modal Logic and First-Order Logic is the *Standard Translation* ($ST$). This translation maps any modal formula into an equivalent FOL formula over a relational signature containing only unary predicates (corresponding to modal propositions) and a single binary predicate $R$ (corresponding to the accessibility relation between worlds).

Let $\mathcal{P}$ be a set of propositional variables. The standard translation $ST_x(\phi)$ of a modal formula $\phi$ evaluated at a world $x$ is defined inductively:

**Definition 2.5** (Standard Translation)**.** *1.* ***Atomic:***
*For any propositional variable $p \in \mathcal{P}$, $ST_x(p) = P(x)$, where $P$ is a unary predicate.*

2. ***Boolean:***
   $ST_x(\neg\phi) = \neg ST_x(\phi)$ *and* $ST_x(\phi \wedge \psi) = ST_x(\phi) \wedge ST_x(\psi)$.

3. ***Existential Modality (Diamond):***
   $ST_x(\Diamond\phi) = \exists y\big(R(x,y) \wedge ST_y(\phi)\big)$.

4. ***Universal Modality (Box):***
   $ST_x(\Box\phi) = \forall y\big(R(x,y) \Rightarrow ST_y(\phi)\big)$.

As evident from the **Existential** and **Universal** rules, the translation of modal operators inherently produces guarded quantifiers. The binary relation $R(x, y)$ acts as a strict local guard, evaluating the sub-formula $ST_y(\phi)$ only in worlds $y$ that are directly accessible from the current world $x$.

### 2.5.2 Tree Representation and Variable Alternation

A remarkable structural property of the Standard Translation is its extreme parsimony with variables. Although the translation of nested modal operators (e.g., $\Diamond\Box\Diamond p$) conceptually moves through a sequence of distinct worlds, the resulting FOL formula does not require an unbounded number of variable names.

From an Abstract Syntax Tree (AST) perspective, building the translation requires tracking the current context node (the active world). When a modal operator expands the AST by introducing a quantifier over a new world, the target variable only needs to be distinct from the immediately preceding active world. Once the traversal moves deeper into the tree, the variables from higher up in the hierarchy are no longer actively referenced in the local relational scope and can be safely overwritten.

Consequently, any modal formula can be translated using exactly two variables—conventionally denoted as $w_1$ and $w_2$—which strictly alternate as the depth of the modal nesting increases. For instance, consider the standard translation of the modal formula $\Box(p \vee \Diamond q)$ evaluated at an initial world $w_1$, as shown in Equation (2.6):

$$\forall w_2\Big(R(w_1, w_2) \rightarrow \big(P(w_2) \vee \exists w_1(R(w_2, w_1) \wedge Q(w_1))\big)\Big) \tag{2.6}$$

As illustrated in Figure 2.5, this variable alternation relies on local rebinding rather than an expanding variable stack. This structural behavior demonstrates why standard Modal Logic can be naturally embedded into $FO^2$: it relies exclusively on a two-variable pool with recycling. Furthermore, because its quantifiers are always structurally shielded by the accessibility relation $R$, it perfectly embeds into the Guarded Fragment (GF). Finally, because its propositional properties are strictly unary ($P(x)$ and $Q(x)$) and are evaluated locally at the context nodes, it inherently satisfies the restrictions of the Unary Negation Fragment (UNFO).

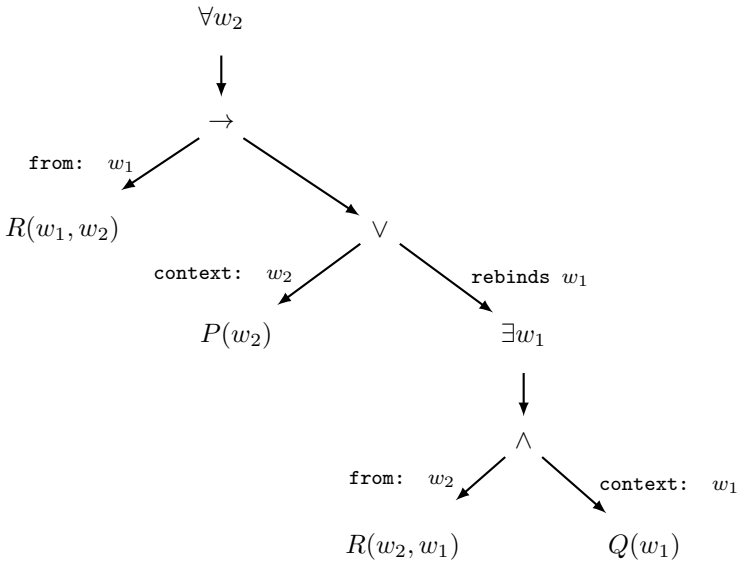


Figure 2.5: Syntactic tree of the translated modal formula (2.6). The translation strictly alternates between $w_1$ and $w_2$. When the inner diamond operator ($\Diamond q$) is expanded, it reuses $w_1$ for its target world, completely shadowing the original initial world.

### 2.5.3 Summary

In conclusion, Propositional Modal Logic serves as the central node connecting all the fragments studied in this chapter.

- The **Two-Variable Fragment** generalizes ML by abandoning the guard relation entirely, relying instead on the strict two-variable limit and recycling.
- The **Fluted Fragment** generalizes ML by viewing the sequence of modal transitions as a strict variable-ordering discipline (the scope stack). Crucially, unlike GF, FF allows the negation of relational atoms, enabling the embedding of enriched modal logics such as Boolean modal logic.
- The **Guarded Fragment** generalizes ML by maintaining the strict guard mechanism but extending it to relations of arbitrary arity.
- The **Unary Negation Fragment** generalizes ML by lifting restrictions on positive quantification entirely, demanding only that negation remains structurally simple (unary).

By understanding the standard translation, we obtain a clear topological map of the boundaries of decidability in First-Order Logic. Modal logic acts as the minimal intersection point, while $FO^2$, FF, GF, and UNFO represent different structural paths to extend its expressivity without crossing the threshold into undecidability. This provides the necessary theoretical foundation for the generation algorithms proposed in this thesis.

# Chapter 3

# System Architecture

Developing an automated generator for complex logical fragments requires more than just theoretical soundness; it demands a robust, scalable, and highly maintainable software architecture. This chapter details the engineering principles, structural design, and implementation strategies underlying the developed system.

To adhere to professional software engineering standards and ensure long-term extensibility throughout the software development lifecycle, the system's architecture heavily leverages established object-oriented design patterns—such as the Factory, Template Method, Context Object, and Visitor patterns. By strictly enforcing the separation of concerns, safe memory management, and modularity, the design guarantees that the generated formulas are not only syntactically well-formed but also mathematically valid by construction.

The following sections systematically explore the core layers of the application. First, we discuss the lexical abstraction and the foundational data structures based on the Abstract Syntax Tree (AST). Next, we analyze the core generation engine and its algorithmic orchestration. Finally, we detail the semantic evaluation pipeline and the specific models used to guarantee formula satisfiability.

## 3.1 Lexical Abstraction

The foundational layer of the architecture handles the definition of the system's logical alphabet. Every syntactic element is encapsulated within the `Symbol` structure and strictly typed through the `SymbolType` enumeration. This system systematically defines all standard logical constructs:

punctuation (( , )), negation ($\neg$), connectives ($\wedge$, $\vee$, $\rightarrow$), quantifiers ($\forall$, $\exists$), equality ($=$), variables ($x_1, x_2$, ..), and predicates ($A_1$ for unary predicates, $B_1$ for binary predicates, ..).

To instantiate these entities, the creation of symbols is delegated to dedicated static **factory methods**, such as `Symbol::var()` for variables, or `Symbol::pred()` for predicates. The implementation of this creational pattern guarantees encapsulation and the enforcement of domain invariants right at the moment of creation.

The internal representation is built upon an Abstract Syntax Tree (AST). This hierarchical data structure intrinsically ensures that generated formulas are well-formed *by construction*.

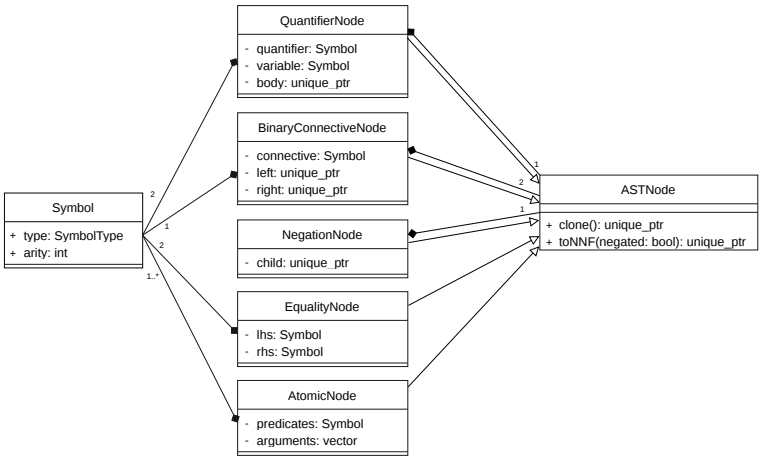


Figure 3.1: Class Diagram representing the AST architecture. The abstract base class `ASTNode` acts as the root, with specific final classes implementing the concrete nodes. Composition relationships highlight how nodes manage their children through exclusive ownership.

The tree's structural integrity relies on strict ownership and safe memory management. Complex nodes (such as `NegNode`, `BinaryConnNode`, and `QuantifierNode`) encapsulate their children via exclusive smart pointers (`std::unique_ptr`). This design perfectly mirrors the compositional nature of logical formulas: parents exclusively own their branches, guaranteeing automatic memory cleanup upon root destruction without the runtime overhead of a garbage collector.

This robust memory model synergizes with pure polymorphism to execute complex logical transformations, such as the conversion to **Negation Normal Form** (`toNNF()`). Relying on polymorphic recursion, each node independently applies type-specific rules to dynamically generate a structurally modified branch. For instance, an implication within a `BinaryConnNode` dynamically rewrites itself into an equivalent disjunction (`OR`) with a negated left child, seamlessly propagating the transformation down its exclusively owned subtrees.

## 3.2 Formula Generation

Once the logical alphabet and the structural foundation (AST) are defined, the system requires a robust engine to dynamically assemble these components into well-formed logical expressions. This responsibility is delegated to the `FormulaBuilder`, a core component designed specifically to orchestrate the generation of the Abstract Syntax Tree.

From an architectural standpoint, this component is driven by the **Template Method** design pattern to balance shared orchestration logic with strict fragment-specific syntactic rules. The abstract base class `FormulaBuilder` establishes the invariant algorithmic skeleton: it handles the recursive descent mechanism, the retry loops, and the pre-processing configurations. However, it intentionally defers the instantiation of leaf nodes and the logic for satisfiability to its derived classes through pure virtual methods, specifically `buildAtomic()`, `generateSAT()`, and `buildComponentUNSAT()`.

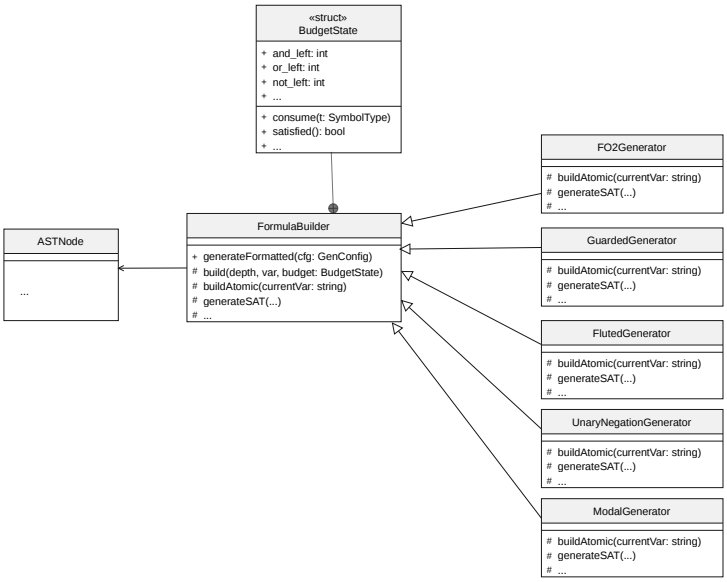


Figure 3.2: UML Class Diagram illustrating the Template Method pattern. `FormulaBuilder` defines the algorithmic skeleton, while concrete generator classes implement the abstract hooks to enforce their specific syntactic rules without duplicating the orchestration logic.

As illustrated in the class diagram (Figure 3.2), concrete implementations—such as the `FO2Generator`, `GuardedGenerator`, and `ModalGenerator`—act as specific strategies, injecting their domain-specific rules into these structural hooks. This polymorphic approach guarantees that each logical fragment can rigorously

enforce its own restrictions without altering the central tree-building engine.

To manage the complexity of the recursive generation without heavily polluting method signatures, the architecture implements the **Context Object** pattern through a nested `BudgetState` structure. This state object encapsulates a mutable context for a single building session, dynamically tracking the remaining allowance for connectives and quantifiers. The state semantics are strictly defined: a value of -1 indicates an unconstrained resource, 0 completely forbids the usage of a specific symbol, and any value greater than 0 represents a limited resource that is meticulously decremented down the recursive call stack.

Because logic generation with exact structural constraints can be mathematically unpredictable, the system is engineered for high resilience. The main entry point, `generateFormatted()`, wraps the entire generation process in a retry loop protected by a try-catch mechanism. If a partially generated AST violates the strict limits defined in the `BudgetState`, the local scope throws a custom `BudgetRetryException`. The orchestrator immediately catches this exception, safely discards the invalid tree, and restarts the generation process up to a predefined threshold. This design guarantees that the final output either perfectly satisfies the configuration object constraints or gracefully terminates with an `std::invalid_argument` exception if the requested shape is fundamentally unsatisfiable.

## 3.3 The FO2 SAT Model and Visitor

To generate logical formulas that are strictly satisfiable (SAT) within the two-variable fragment (FO2), the system's architecture must transition from purely syntactic assembly to active semantic evaluation. This complex responsibility is orchestrated by the `FO2SATGenerator`, which extends the base `FO2Generator` to inject mathematical validation directly into the generation pipeline[cite: 10]. From a software design perspective, this process relies on the strict separation of concerns between the structural representation of the formula, the mathematical domain, and the evaluation algorithm.

The mathematical ground truth required for semantic validation is encapsulated within the `FiniteModel` class[cite: 11]. Operating as a Domain Model, this component dynamically generates and stores randomized boolean interpretations for both unary and binary predicates over a specified finite domain size[cite: 11]. By abstracting the domain logic, the model provides a clean, stateless interface (`evalAtom()`) to query the truth value of any given assignment without exposing its internal combinatorial matrices[cite: 11].

The most critical architectural challenge in this phase is traversing and evaluating the Abstract Syntax Tree without polluting the pure data structure of the nodes with domain-specific semantic logic. The architecture elegantly resolves this by implementing the **Visitor Pattern**[cite: 8]. The `FO2Evaluator` class acts as the concrete visitor, realizing the `ASTVisitor` interface to enable double-dispatch traversal across the tree[cite: 8]. As it visits specific node types—such as `AtomicNode`, `QuantifierNode`, or `BinaryConnNode`—the evaluator dynami-

cally computes their boolean truth values against the current variable assignment and the referenced `FiniteModel`[cite: 8].

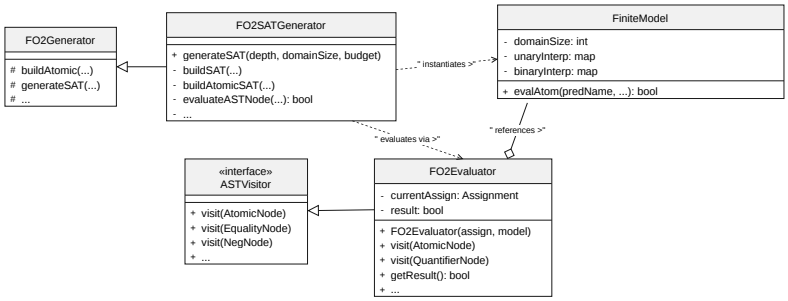


Figure 3.3: UML Class Diagram detailing the semantic SAT generation architecture. The `FO2SATGenerator` orchestrates the `FiniteModel`. It heavily relies on the **Visitor Pattern** via the `FO2Evaluator` to safely traverse the AST and compute semantic truth values without altering the underlying node structures.

As depicted in Figure 3.3, the `FO2SATGenerator` acts as the central coordinator of these patterns[cite: 10]. During the recursive construction of a formula, it acts as a client that proposes candidate sub-trees and immediately validates them by instantiating an `FO2Evaluator`[cite: 9, 10]. If a generated branch does not satisfy the semantic targets dictated by the `FiniteModel`, the generator leverages this immediate feedback to dynamically adjust the structure, such as wrapping the branch in a `NegNode` or falling back to a tautology, ensuring that the final output is structurally sound and mathematically satisfiable by construction[cite: 9].

## 3.4 External Verification: The Vampire Actor

While the internal generation engine handles the rigorous construction of syntactically valid formulas, ensuring the satisfiability (SAT) of complex fragments—such as the Fluted fragment—often requires a "generate-and-test" approach. Within this system's architecture, the Automated Theorem Prover (ATP) **Vampire** is modeled as an independent, out-of-process *Actor* that serves as an external semantic oracle.

Rather than tightly coupling the solver's complex inference logic to the generation engine, the architecture interfaces with this external actor through a dedicated proxy component: the `VampireRunner` class. This component acts as an adapter bridging the internal C++ application with the external operating system environment. Its primary responsibilities include:

- **Data Serialization:** Receiving the internally generated AST and formatting it into the standard TPTP (Thousands of Problems for Theorem Provers) language.

- **Process Orchestration:** Safely managing temporary file I/O and spawning a separate OS-level sub-process to execute the Vampire binary with strict time and memory limits.
- **Result Parsing:** Capturing the standard output via pipes and extracting the standard SZS status (e.g., *Satisfiable*, *Unsatisfiable*, *Timeout*) to feed the result back into the application's control flow.

By treating Vampire as an external actor, the architecture maintains strict modularity and separation of concerns. The generation pipeline can rapidly produce candidate formulas (e.g., in "free" mode) and immediately delegate the heavy lifting of semantic verification to the solver, which filters out invalid instances.


Figure 3.4: UML Component Diagram illustrating the integration of the Vampire Actor. The `VampireRunner` class acts as the structural bridge between the internal generation pipeline and the external ATP process, parsing the SZS output to validate formula satisfiability.

While this section defines Vampire's structural role and architectural integration as a verification oracle, a comprehensive theoretical analysis of its

underlying logical calculus, resolution techniques, and its central role in the dedicated testing framework will be thoroughly discussed later in Chapter 5.

## 3.5 Architecture Overview

Having detailed the individual structural and behavioral components of the system, this section synthesizes the global software architecture. The application operates as a unidirectional pipeline characterized by a strict separation of concerns among the configuration layer, the generation engine, and the mathematical evaluation domain.

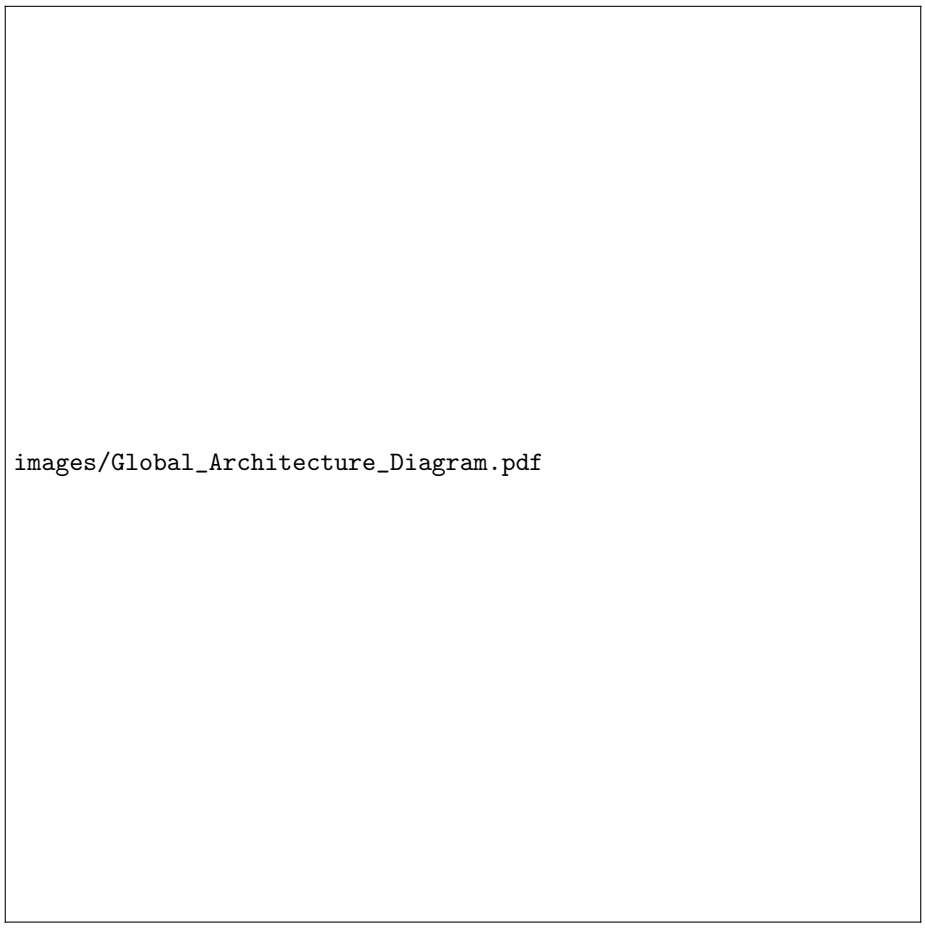


Figure 3.5: Global System Architecture. The diagram illustrates the complete data flow: from the CLI configuration down to the Template Method orchestrator, integrating the semantic SAT validation via the Visitor pattern on the generated AST.

As illustrated in the global architecture diagram (Figure 3.5), the execution flow begins at the CLI, which configures the execution environment. Depending on the requested fragment, a specific concrete generator (e.g., `FO2SATGenerator`) is instantiated. This generator utilizes the factory methods to spawn `Symbol` instances and begins recursively assembling the AST.

When mathematical validation is required (e.g., enforcing SAT models), the generator interacts with the `FiniteModel` and uses the `FO2Evaluator` to traverse the partial AST via double-dispatch. If the validation succeeds, the tree solidifies; if it fails, the local budget triggers a structural adjustment or a controlled retry mechanism. Ultimately, the finalized AST is polymorphicly resolved into

a formatted string representation and returned to the standard output, ready for external testing and verification procedures.

## 3.6 System Interface and CLI

While the core generation engine handles the rigorous construction of logical formulas, the system requires a robust presentation layer to interact with the end user and external tools. This is achieved through a Command Line Interface (CLI) that acts as the entry point and configuration orchestrator for the entire pipeline.

The CLI is designed to decouple user input from the internal generation logic. When a generation command is issued, the interface parses various execution arguments—such as the targeted logical fragment (e.g., FO2, Guarded, Fluted), the maximum abstract syntax tree depth, the random seed for reproducibility, and specific structural constraints. These inputs are not passed raw to the internal logic; instead, they are sanitized and encapsulated into a structured `GenConfig` object.

This configuration object initializes the `BudgetState`, directly influencing the recursive generation process. Furthermore, the CLI strictly formats the generated formulas into standard logical notation via the `generateFormatted()` entry point. By outputting well-formed strings to standard output or safely routing them to dedicated output directories, the interface guarantees seamless interoperability with external Automated Theorem Provers (ATPs). For instance, the system's output can be directly piped into solvers like Vampire in "free" mode to perform external validation and benchmark the satisfiability of the generated instances.

### 3.6.1 Application Controller

Beyond simple parameter parsing, the main entry point serves as an Application Controller that orchestrates the entire verification and testing lifecycle. Depending on the parsed flags, the CLI dynamically routes the execution flow into distinct operational modes:

- **Standard Generation:** Instantiates the specific fragment generator (e.g., `FO2Generator`, `FlutedGenerator`) and executes the generation loop, actively managing file I/O and retrying up to a maximum threshold if the requested budget constraints fail during AST construction.
- **Dataset Orchestration:** Triggers the `generateDatasets` routine to automatically construct massive, categorized corpora of logical formulas.
- **Automated Benchmarking:** Invokes the benchmarking and timeout analysis workflows, seamlessly passing the `VampireRunner` instance to evaluate previously generated datasets.

### 3.6.2 Self-Documenting Interface

To ensure professional usability and ease of integration into larger automated workflows, the CLI is fully self-documenting. Invoking the application with the `--help` flag provides a comprehensive overview of the available commands, configuration limits, and structural options. A conceptual snippet of this interface is illustrated in Figure 3.6.

Command Line Interface

```
USAGE: ./DecidableFragmentsGen <options>

BASIC OPTIONS:                          FORMULA STRUCTURE OPTIONS:
  --fragment <name>                       --and <n|min:max>
  --mode <mode>                           --or <n|min:max>
  --domain-size <n>                       --not <n|min:max>
  --depth <n>                             --exists <n|min:max>
  --count <n>                             --forall <n|min:max>
  --seed <n>                              --implies <n|min:max>
  --help                                  --eq <n|min:max>

TESTING & DATASETS OPTIONS:             VOCABULARY OPTIONS:
  --run-benchmarks <path>                 --preds <count>/<arity>
  --generate-datasets                     --arity <k|mixed>
  --run-timeout-analysis
                                        TRANSFORMATION OPTIONS:
OUTPUT OPTIONS:                           --transform <mode>
  --output <format>
                                        VERIFICATION OPTIONS:
                                          --verify
                                          --vampire-path <path>
                                          --vampire-timeout <n>
```

Figure 3.6: The compact Command Line Interface help menu, displaying the configuration options grouped by functional category.

# Chapter 4

# Implementation

The core contribution of this thesis is the design and implementation of a modular, parametric system for generating random formulae that belong to specific decidable fragments of First-Order Logic. Generating structured logical formulae is a non-trivial task; purely random syntactic string generation frequently results in ill-formed or trivially unsatisfiable expressions. To overcome this, our implementation relies on a highly configurable, probabilistic architecture centered around a robust internal data structure.

## 4.1 Development Environment

The development of the formula generation system commenced in February 2026. Given the computational intensity of generating deeply nested logical structures and the need for strict deterministic memory management, the project was implemented in **C++**, utilizing the most modern standard available at the time of inception: **C++23**.

The choice of C++23 allowed the codebase to benefit from modern programming paradigms, prioritizing safety without sacrificing zero-cost abstractions. Several key modern C++ features were heavily utilized throughout the codebase:

- **Smart Pointers for Memory Safety:** Raw pointers were strictly avoided. The standard library's `<memory>` header, specifically `std::unique_ptr`, was employed to manage the lifecycle of polymorphic objects, guaranteeing the absence of memory leaks during the recursive generation of trees.
- **Move Semantics (`std::move`):** To avoid expensive deep copies of tree branches during AST construction, `std::move` was consistently used to transfer ownership of generated sub-formulas between nodes and functions.
- **Modern Attributes:** The `[[nodiscard]]` attribute was extensively applied to pure virtual interfaces and builder methods (e.g., in the `FormulaBuilder`

class) to ensure that generated sub-trees or vital state queries were not accidentally ignored by the compiler.

- **Lambda Expressions:** Localized algorithmic logic, such as the final formatting of formulas into TPTP syntax or forced quantification steps, was encapsulated in stateful lambdas (e.g., `auto finalize = [&](...)`), improving code locality and readability.
- **Exception Handling for Control Flow:** Custom exception types, such as `BudgetRetryException`, were implemented to elegantly backtrack during randomized generation if a specific architectural constraint (or budget) could not be met.

The project uses `CMake` as its build system, ensuring cross-platform compilation compatibility with modern toolchains (GCC, Clang, or MSVC) capable of supporting the C++23 standard.

## 4.2 The Abstract Syntax Tree (AST)

The fundamental data structure of the system is the Abstract Syntax Tree (AST), which translates the formal grammar of First-Order Logic into a strictly typed C++ class hierarchy. As requested by the architectural design, the implementation relies heavily on modern C++ features—primarily `std::unique_ptr`—to ensure exclusive memory ownership and automatic bottom-up deallocation without garbage collection overhead.

### 4.2.1 Node Hierarchy and Interfaces

The tree is built around the pure virtual base class `ASTNode`. This class exposes a unified interface that all concrete logical nodes (`AtomicNode`, `EqualityNode`, `NegNode`, `BinaryConnNode`, and `QuantifierNode`) must implement:

- `clone()`: Performs a recursive deep copy of the tree branch.
- `toString()` and `toTPTP()`: Handle the string serialization of the formula. The latter maps the internal AST representation to the standard TPTP format (e.g., translating existential quantifiers to `?` and universal to `!`, while ensuring variables are correctly capitalized).
- `accept(ASTVisitor&)`: Acts as the entry point for the Visitor design pattern, allowing external components (like SAT model generators) to traverse the AST without modifying the node classes.

### 4.2.2 The Negation Normal Form (NNF)

To seamlessly interface with external Theorem Provers, the AST includes a built-in transformation to Negation Normal Form (NNF). Instead of relying on an external visitor, the conversion is handled polymorphically.

The procedure recursively eliminates implications and pushes negations down to the atomic leaves using De Morgan's laws, gracefully flipping binary connectives and quantifiers on the fly. The underlying logic is formalized in Algorithm 1.

**Algorithm 1:** NNF Transformation for AST Nodes

**function** toNNF*(n: Node, neg: $\mathbb{B}$): Node*
  **if** $n = (L \to R)$ **then**
    $n' \leftarrow (\neg L \lor R)$;
    **return** toNNF($n'$, *neg*);
  **else if** $n = (L \circ R)$ **with** $\circ \in \{\land, \lor\}$ **then**
    $\circ' \leftarrow \circ$;
    **if** *neg* = **true then**
      $\circ' \leftarrow$ dual($\circ$) ; `// dual`$(\land) = \lor, $`dual`$(\lor) = \land$
    **end**
    **return** (toNNF($L$, *neg*) $\circ'$ toNNF($R$, *neg*));
  **else if** $n = (\mathcal{Q}x.\, \phi)$ **with** $\mathcal{Q} \in \{\forall, \exists\}$ **then**
    $\mathcal{Q}' \leftarrow \mathcal{Q}$;
    **if** *neg* = **true then**
      $\mathcal{Q}' \leftarrow$ dual($\mathcal{Q}$);
    **end**
    **return** ($\mathcal{Q}'x.$ toNNF($\phi$, *neg*));
  **else if** $n = (\neg\phi)$ **then**
    **return** toNNF($\phi, \neg$*neg*);
  **else**
    `// Atomic Leaf`
    **if** *neg* = **true then return** $\neg n$;
    **else return** $n$;
  **end**

## 4.3 The Probabilistic Budget Model

The generation of logically valid and structurally interesting formulas requires more than just adhering to fragment syntax; it requires a mechanism to steer the random generation toward specific user-defined shapes (e.g., forcing exactly 10 conjunctions and no existentials). The `FormulaBuilder` base class acts as the central engine for this probabilistic generation [cite: 14].

### 4.3.1 Constraint Management and Backtracking

The generation process is governed by the `BudgetState` structure, which keeps track of operational counters for each logical symbol (connectives and quantifiers) during a single tree-building session [cite: 14]. The counters operate on a tri-state semantic:

- **Free ($-1$):** The symbol can be used infinitely [cite: 14].

- **Forbidden ($0$):** The symbol is completely excluded from the generation [cite: 14].
- **Constrained ($n > 0$):** The symbol must be used exactly $n$ times [cite: 14]. Each insertion decrements the counter [cite: 14].

Because generation is inherently stochastic, a randomly built branch might terminate prematurely (e.g., reaching maximum depth) before exhausting the constrained budget. To solve this, the generator wraps the building process in a backtracking loop: if a tree is completed but the `BudgetState` is not fully satisfied, the system throws a `BudgetRetryException`, discards the AST, and retries up to a `MAX_RETRY` limit of 500 attempts [cite: 13, 14].

## 4.4 Applied Probability Distributions

The generation pipeline relies on a deterministic pseudo-random number generator (PRNG), initialized via a user-defined seed, to traverse the mathematical search space[cite: 3]. The system's randomness is abstracted by the `randInt(lo, hi)` utility, which wraps the C++ `std::uniform_int_distribution` to systematically model discrete probability distributions[cite: 3].

- **Uniform Selection ($P(X = x) = 1/N$):** When navigating the AST construction, the engine continuously selects the next logical operator or atomic predicate from a dynamic pool of valid candidates. A uniform distribution guarantees that, without external budget constraints, every structurally admissible symbol is equally likely to be instantiated[cite: 3].
- **Fair Bernoulli Trials ($p = 0.5$):** Binary decisions without inherent bias are modeled as fair coin tosses. For instance, when resolving a symmetric decision fork—such as randomly selecting between a universal or existential quantifier when both are fully admissible—the system relies on a 50% probability (`randInt(0, 1) == 0`) to maintain a statistically balanced logic tree[cite: 1, 3].
- **Weighted Bernoulli Trials and the 85% Heuristic ($p = 0.85$):** The system dynamically steers the random search space when strict budgets are applied. To maximize the success rate of the backtracking loop, the `pickType` method introduces a probabilistic bias. If there are "forced" symbols (those with a remaining budget $n > 0$), the generator selects from this restricted pool with an **85% probability**. This biased trial, computationally evaluated as `randInt(1, 100)` $\leq 85$, gently guides the generation toward budget completion without collapsing into pure determinism[cite: 3].

### 4.4.1 UNSAT Generation Algorithm

Generating unsatisfiable (UNSAT) formulas while strictly respecting a given structural budget presents a unique mathematical challenge. The framework

employs a structural contradiction approach: generating a formula in the form $\Phi = \phi \wedge \neg\phi'$ (where $\phi'$ is an exact structural clone of $\phi$ but with distinct variables depending on the fragment rules) [cite: 13].

To ensure the final AST ($\Phi$) exactly matches the user's requested budget, the `FormulaBuilder` must mathematically divide the budget before recursively generating the sub-component $\phi$ [cite: 13]. This requires the initial budget for `AND` and `NEG` to be odd (since the root connects the two branches with one `AND` and one `NEG`), while all other operators must be strictly even [cite: 13]. The logic is formalized in Algorithm 2.

**Algorithm 2:** Budget-Constrained UNSAT Generation

**function** generateUNSAT*(d: Depth, B: Budget): Node*
  **if** $B$.AND is even $\vee$ $B$.NEG is even **then throw** Error;
  **if** $\exists x \notin \{\text{AND}, \text{NEG}\} \mid B.x$ is odd **then throw** Error;
  `// Consume root connectors`
  $B$.consume(AND);
  $B$.consume(NEG);
  `// Split remaining budget strictly in half`
  $B_\phi \leftarrow \{x \mapsto \lfloor B.x/2 \rfloor \mid x \in B\}$;
  `// Generate left branch and clone for right branch`
  $\phi \leftarrow$ buildComponentUNSAT$(d-1, B_\phi)$;
  $\phi' \leftarrow$ clone$(\phi)$;
  `// Calculate true consumption`
  **foreach** $x \in B$ **do**
    $\Delta_x \leftarrow (B.x/2) - B_\phi.x$ ; `// Actual nodes used by` $\phi$
    $B.x \leftarrow \max(0, B.x - (\Delta_x \times 2))$;
  **end**
  **return** $(\phi \wedge \neg\phi')$;

## 4.5 Fragment-Specific Constraints

While the probabilistic budget model dictates the overarching shape of the formulas, ensuring that the generated AST conforms to the deep semantic and syntactic restrictions of specific logical fragments requires dedicated algorithms. For most fragments, generating a satisfiable (SAT) formula relies on a rejection-sampling approach: formulas are generated freely within the syntactic constraints, and an external theorem prover (like Vampire) is invoked to filter out the unsatisfiable ones.

However, for the Two-Variable Fragment ($\text{FO}^2$), the system implements a highly optimized, *correct-by-construction* SAT generation engine.

### 4.5.1 The Two-Variable Fragment ($\mathrm{FO}^2$) and SAT Construction

The standard $\mathrm{FO}^2$ generation (`FO2Generator`) inherently guarantees the two-variable restriction by toggling a single active variable state ($V_1 \leftrightarrow V_2$) during the recursive descent, thus eliminating the need for complex variable binding environments [cite: 16, 17].

To natively generate SAT instances without relying on external provers, the system introduces the `FO2SATGenerator` [cite: 19, 20]. This generator shifts the paradigm from purely syntactic generation to *model-driven generation*.

#### Model-Driven Target Propagation

The core idea is to first instantiate a random `FiniteModel` over a small domain (e.g., $\{0, 1, 2\}$) and then recursively build an AST that evaluates to `true` under a specific set of target assignments (`Targets`) [cite: 19]. To verify truth values during generation, the system employs the `FO2Evaluator`, a specialized visitor that evaluates AST nodes against the finite model using standard logical semantics [cite: 18].

As the tree is built downward, the `Targets` are algebraically propagated and manipulated depending on the logical connective chosen, as formalized in Algorithm 3.

**Algorithm 3:** Target Propagation for $\mathrm{FO}^2$ SAT Generation

**function** buildSAT*(d: Depth, T: Targets, M: Model, v: Var): Node*
  **if** $d = 0$ **then return** buildAtomicSAT$(T, M, v)$ [cite: 19];
  $op \leftarrow$ pickType$(d)$ [cite: 19];
  **if** $op =$ AND **then**
    `// Both branches must satisfy all targets`
    **return** (buildSAT$(d-1, T) \wedge$ buildSAT$(d-1, T)$) [cite: 19];
  **else if** $op =$ OR **then**
    `// Randomly distribute targets to left and right branches`
    $(T_L, T_R) \leftarrow$ split$(T)$ [cite: 19];
    **return** (buildSAT$(d-1, T_L) \vee$ buildSAT$(d-1, T_R)$) [cite: 19];
  **else if** $op =$ NEG **then**
    `// The child must satisfy the complement of the targets`
    $T^C \leftarrow$ complementTargets$(M, T, v)$ [cite: 19];
    **return** $\neg$buildSAT$(d-1, T^C)$ [cite: 19];
  **else if** $op \in \{\exists, \forall\}$ **then**
    `// Expand targets across the domain elements`
    $T' \leftarrow$ expandTargets$(T, M, op)$ [cite: 19];
    **return** $(\mathcal{Q}\bar{v}.$ buildSAT$(d-1, T'))$ [cite: 19];

#### Sparse Complement Optimization

A significant computational bottleneck in target propagation arises during negation (`NEG`) and implication (`IMPLIES`), which require computing the complement of the current targets within the domain space [cite: 19]. A naive multi-dimensional array representation for domain assignments would result in severe memory explosion for larger domains.

To overcome this, the `complementTargets` procedure utilizes a sparse memory approach [cite: 19]. Instead of allocating the full Cartesian product, the algorithm hashes the active assignments into a 1D unique key (e.g., $e_1 \times |D| + e_2$) and stores them in an `unordered_set` [cite: 19]. The complement is then rapidly generated by querying this hash map and capped at a `MAX_TARGETS` limit of 1000 to maintain constant-time bounds during deep recursions [cite: 19].

#### Leaf Resolution

At the bottom of the tree ($d = 0$), `buildAtomicSAT` serves as the anchor point. It iterates through the randomized vocabulary and checks the `FiniteModel` to find a predicate that is uniformly `true` (or uniformly `false`) for *all* assignments in the current `Targets` [cite: 19]. If it finds a uniformly true predicate, it returns the atomic node; if uniformly false, it wraps the atom in a `NegNode` [cite: 19]. This ensures the constructed formula perfectly matches the propagated truth requirements of the root model [cite: 19].

### 4.5.2 The Fluted Fragment (FL): Scope Stack and Forced Quantification

The Fluted Fragment imposes strict ordering constraints on predicate arguments, requiring them to follow a sequence dictated by the quantifier scope. The `FlutedGenerator` class ensures these constraints natively during the AST construction [cite: 22, 23].

#### Scope Stack Abstraction

Instead of maintaining a complex data structure for the scope stack, the implementation abstracts it using a simple integer, `stackDepth` [cite: 22]. As the tree generation descends through `EXISTS` or `FORALL` nodes, `stackDepth` is incremented, and new variables are systematically generated with the naming convention $x_n$ (e.g., $x_1, x_2, \dots$) [cite: 22].

When generating an atomic leaf at a given `stackDepth`, the `buildAtomicFL` method sequentially assigns the arguments [cite: 22]. For a predicate of arity $k$ and a current stack depth $m$, the arguments are strictly assigned as $(x_{m-k+1}, \dots, x_m)$ [cite: 22]. This logic is centralized in the `predArgNames` utility, ensuring that no invalid variable permutations can occur [cite: 22, 23]. Additionally, the generation of equality nodes ($x_{m-1} = x_m$) is statically prevented by the `candidateTypesFL` method if the current `stackDepth` is less than 2 [cite: 22].

**Forced Quantification Heuristic**

A unique challenge in randomly generating $\mathcal{FL}$ formulas arises when the generation budget dictates the creation of a leaf node (i.e., `depth == 0`), but the active vocabulary lacks predicates that fit the current `stackDepth`. Specifically, the `admissiblePreds` function filters out any predicate whose arity exceeds `stackDepth` [cite: 22, 23].

To prevent generation deadlocks, the algorithm introduces a `forcedQuant` fallback closure [cite: 22]. If no admissible predicates are available, the generator forcefully consumes an available quantifier from the budget, increments the `stackDepth`, and recursively attempts to build the body [cite: 22]. This mechanism guarantees that the generation process will "dig" deep enough to eventually satisfy the arity requirements of the vocabulary, as outlined in Algorithm 4.

**Algorithm 4:** $\mathcal{FL}$ Leaf Generation with Forced Quantification

**function** buildFL*(d: Depth, s: StackDepth, B: Budget): Node*
  $A \leftarrow$ admissiblePreds($s$) ; `// Predicates with arity` $\leq s$ `[cite: 22]`
  **if** $d = 0 \vee$ candidatesEmpty() **then**
    **if** $A \neq \emptyset$ **then**
      **return** buildAtomicFL($s$) [cite: 22];
    **else**
      `// Force quantifier to increase stack depth`
      **if** $B$.EXISTS $= 0 \wedge B$.FORALL $= 0$ **then throw** BudgetRetryException [cite: 22];
      $\mathcal{Q} \leftarrow$ pickAvailableQuantifier($B$) [cite: 22];
      $B$.consume($\mathcal{Q}$) [cite: 22];
      $x_{\text{new}} \leftarrow$ varName($s + 1$) [cite: 22];
      *body* $\leftarrow$ buildFL($\max(0, d - 1), s + 1, B$) [cite: 22];
      **return** ($\mathcal{Q}x_{\text{new}}$. *body*) [cite: 22];
    **end**
  `...// Standard recursive generation for connectives`

### 4.5.3 The Guarded Fragment (GF): Guard Generation and Scope Tracking

The Guarded Fragment restricts quantification by mandating that any quantified sub-formula must be shielded by an atomic formula (the guard) containing all free variables in the current scope. To enforce this, the `GuardedGenerator` actively maintains a state vector, `currScopeFreeVars`, representing the variables $\bar{x}$ that are free in the current recursive step [cite: 26].

**On-the-fly Guard Synthesis**

When the generation algorithm selects a quantifier (e.g., an existential node), it cannot simply bind new variables $\bar{y}$. It must simultaneously guarantee the existence of a valid guard predicate $\alpha(\bar{x}, \bar{y})$ whose arity is exactly equal to the total number of free and newly bound variables ($|\bar{x}| + |\bar{y}|$) [cite: 25].

The system queries the active vocabulary via the `admissibleGuards` method to find matching predicates [cite: 25]. If no such predicate exists, the quantifier choice is dynamically rejected via a `BudgetRetryException` [cite: 25]. If a guard is found, the system updates the scope, builds the body, and wraps the structural components logically (using $\wedge$ for $\exists$, and $\rightarrow$ for $\forall$), as formalized in Algorithm 5.

**Algorithm 5:** GF Existential Guard Generation

**function** buildGuardedExists*(d: Depth, B: Budget, $\bar{x}$: ScopeVars): Node*
  $k_{\max} \leftarrow \min(3, d)$ [cite: 25];
  $V_k \leftarrow \{k \in [1, k_{\max}] \mid \text{admissibleGuards}(|\bar{x}| + k) \neq \emptyset\}$ [cite: 25];
  **if** $V_k = \emptyset$ **then throw** BudgetRetryException [cite: 25];
  $k \leftarrow \text{pickRandom}(V_k)$ [cite: 25];
  $\bar{y} \leftarrow \text{nextVarNames}(k)$ [cite: 25];
  `// Guard arity strictly equals` $|\bar{x}| + |\bar{y}|$
  $\alpha \leftarrow \text{buildGuard}(\bar{x}, \bar{y})$ [cite: 25, 26];
  `// Recursively build body with extended scope`
  $body \leftarrow \text{buildGF}(d - 1, B, \bar{x} \cup \bar{y})$ [cite: 25];
  **return** $(\exists \bar{y}.\, (\alpha \wedge body))$ [cite: 25, 26];

### 4.5.4 The Unary Negation Fragment (UNFO): Dependency Truncation

The Unary Negation Fragment (UNFO) allows full First-Order Logic syntax with one critical restriction: the negation operator ($\neg\phi$) can only be applied to sub-formulas $\phi$ that contain at most one free variable [cite: 28]. The `UnaryNegGenerator` handles this elegantly without requiring complex dependency graphs [cite: 27].

**Negation Scope Reduction**

Similar to the Guarded Fragment, the UNFO generator tracks active variables via a `currFreeVars` vector [cite: 27]. However, the crucial intervention occurs when the random builder selects a `NEG` node [cite: 27]. Instead of passing the entire active scope to the child node, the `buildNegatedBody` method deliberately truncates the scope [cite: 27]. It randomly selects a single variable from `currFreeVars` and passes only that variable downward, effectively enforcing the UNFO rule by construction [cite: 27, 28].

**Algorithm 6:** UNFO Negation Scope Truncation

```
function buildNegatedBody(d: Depth, B: Budget, x̄: ScopeVars): Node
    V_unary ← ∅ [cite: 27];
    if x̄ ≠ ∅ then
        v ← pickRandom(x̄) [cite: 27];
        V_unary ← {v} [cite: 27];
    end
    // Body is generated with at most ONE free variable
    body ← buildUN(d − 1, B, V_unary) [cite: 27];
    return ¬body [cite: 27];
```

**Scope Injection via Forced Quantification**

A secondary consequence of the UNFO syntax is that a formula might reach a leaf node with an empty `currFreeVars` state (e.g., at the very root of the AST before any quantifiers are generated) [cite: 27]. If the active vocabulary only contains predicates of arity $\geq 1$, generating a valid atom becomes impossible [cite: 27]. To resolve this, the system employs a `forcedQuant` heuristic [cite: 27]. If no variables are in scope, it bypasses the standard connective randomizer, forcefully consumes a quantifier from the budget, and recursively injects a new variable into the scope, ensuring that atomic leaves can always be populated correctly [cite: 27].

### 4.5.5 Propositional Modal Logic and the Standard Translation

As introduced in the theoretical background, Propositional Modal Logic (ML) can be embedded into First-Order Logic via the *Standard Translation*. Since ML is a syntactic fragment of $FO^2$, the implementation of the `ModalGenerator` borrows the variable recycling concept (toggling strictly between world variables `w1` and `w2`) [cite: 31], but requires a specific structural override for quantifiers [cite: 30].

The generator intercepts the instantiation of `EXISTS` and `FORALL` nodes (which semantically correspond to the modal operators $\Diamond$ and $\Box$, respectively) [cite: 30]. Instead of generating standard FOL quantifiers, it structurally injects the accessibility relation $R(w_i, w_j)$ [cite: 30]:

**Algorithm 7:** Standard Translation in `ModalGenerator`

```
1  function buildModal(d: Depth, w: CurrentWorld, B: Budget): Node
2      op ← pickType(d, B) [cite: 30];
3      if op ∈ {∃, ∀} then
4          w′ ← nextVar(w) ;     // Toggles between w1 and w2 [cite:
            31]
5          α_R ← R(w, w′) ;     // Accessibility relation [cite:  30]
6          φ ← buildModal(d − 1, w′, B) [cite: 30];
7          if op = ∃ then
8              return (∃w′.(α_R ∧ φ)) ;     // Translation of ◇φ [cite:
                30]
9          else
10             return (∀w′.(α_R → φ)) ;   // Translation of □φ [cite:
                30]
11         end
12     ...// Fallback to standard propositional connectives
           [cite:  30]
```

For atomic propositions (the leaves of the tree), the generator restricts the vocabulary to unary predicates $P(w_i)$, representing the propositional variables evaluated at the current world [cite: 30]. This elegantly bridges the gap between modal semantics and the FOL Abstract Syntax Tree [cite: 30].

## 4.6 Output Serialization and TPTP Formatting

Once the generation constraints are met and the Abstract Syntax Tree is fully constructed in memory, the system must serialize the logical formula into a machine-readable text format. While the AST supports standard human-readable stringification via the `toString()` method[cite: 8], the primary target format is the TPTP (Thousands of Problems for Theorem Provers) syntax. This format acts as the *lingua franca* for modern automated theorem provers such as Vampire, E, and SPASS, making it essential for the evaluation pipeline.

### 4.6.1 Polymorphic Translation and Lexical Sanitization

Serialization is handled polymorphically through the `toTPTP()` virtual method, implemented by every concrete node in the AST hierarchy[cite: 8]. During the recursive traversal of the tree, the internal C++ symbolic representations are mapped to TPTP-compliant ASCII operators:

- **Operators:** Logical AND ($\wedge$), OR ($\vee$), IMPLIES ($\rightarrow$), and NEG ($\neg$) are translated to `&`, `|`, `=>`, and `~` respectively[cite: 7]. Universal ($\forall$) and existential ($\exists$) quantifiers are translated using `!` and `?`[cite: 7].

Simultaneously, the AST enforces strict TPTP lexical constraints: variables must begin with an uppercase letter, whereas predicates must begin with a

lowercase letter. Because different fragments use varying variable naming conventions (e.g., `v1`, `x1`, `w1`), the nodes perform dynamic lexical sanitization[cite: 7]. When `AtomicNode::toTPTP()` or `QuantifierNode::toTPTP()` are invoked, predicate names are systematically downcased via standard library utilities, and every variable is processed by the static `tptpVar()` method, which capitalizes the first character[cite: 7]. This built-in sanitization ensures that the generated output remains syntactically valid regardless of the internal fragment's vocabulary.

### 4.6.2 Wrapper Generation and Semantic Roles

The final step of serialization occurs at the root of the `FormulaBuilder`, which encapsulates the generated TPTP string into the standard First-Order Form wrapper structure: `fof(f, role, formula).`[cite: 3].

Crucially, the orchestrator dynamically assigns the semantic *role* based on the requested generation mode[cite: 3]:

- `axiom`**:** Assigned during `FREE` and `SAT` generation modes[cite: 3]. This instructs the external theorem prover to treat the generated formula as a given premise in its knowledge base.
- `negated_conjecture`**:** Assigned during `UNSAT` mode[cite: 3]. Because UNSAT generation produces structurally contradictory formulas ($\phi \wedge \neg\phi'$), tagging them as negated conjectures aligns the synthetic output with standard refutational theorem-proving workflows, optimizing the provers' internal parsing heuristics[cite: 3].

# Chapter 5

# Vampire Theorem Prover

To effectively verify the satisfiability of logically generated fragments, this research relies on **Vampire** [10], a state-of-the-art Automated Theorem Prover (ATP). This chapter outlines the specific theoretical mechanisms Vampire employs to process First-Order Logic (FOL) formulae, execute heuristic searches, and determine satisfiability.

## 5.1 First-Order Clausification

While modern ATPs accept standard First-Order Form (`fof`) syntax, their core reasoning engines require formulae to be strictly structured in Conjunctive Normal Form (CNF). When Vampire receives a `fof` input, it immediately applies an equisatisfiable clausification pipeline [10]:

- **Negation Normal Form:** Implications are expanded and negations are pushed inward.
- **Skolemization:** Existential quantifiers are substituted with Skolem functions.
- **CNF Conversion:** The formula is flattened into a set of clauses (disjunctions of literals).

## 5.2 The Superposition Calculus

Vampire operates via a saturation-based refutation loop driven by the *Superposition Calculus* [2]. This calculus extends standard resolution by inherently handling equality (=) through strict term-reduction orderings (e.g., Knuth-Bendix Ordering). By treating equations as directional rewrite rules (from complex to simple terms), superposition drastically prunes the search space, preventing the infinite loops typical of naive equality substitution.

## 5.3 Portfolio Solving and CASC Mode

Due to the semi-decidable nature of FOL, a single proof strategy is rarely effective across diverse logical fragments. To address this, Vampire implements *Portfolio Solving* [10]. When operating in CASC mode (named after the CADE ATP System Competition), Vampire does not rely on a single heuristic. Instead, it sequentially or concurrently executes a scheduled portfolio of distinct saturation strategies, time-slicing the available execution window to maximize the probability of proof discovery.

## 5.4 Satisfiability and Finite Model Finding

While traditional ATPs excel at proving unsatisfiability (UNSAT) by deriving an empty clause, evaluating decidable fragments often requires confirming satisfiability (SAT). Vampire achieves this through two primary theoretical mechanisms:

- **Saturation Termination:** If the Given Clause algorithm exhausts all possible inferences without generating an empty clause, the clause set is fundamentally satisfiable.
- **Finite Model Finding (FMF):** Vampire employs specialized SAT/SMT-backed engines to actively search for boolean models over finite domains [15].

To efficiently manage complex formulas and model finding, modern versions of Vampire rely on the AVATAR architecture (Advanced Vampire Architecture for Theories and Resolution) [15]. As illustrated in Figure 5.1, AVATAR splits the reasoning process: a dedicated SAT solver handles the propositional boolean structure, while the First-Order engine handles the superposition calculus. If the SAT solver finds a valid boolean model, the prover can successfully halt and classify the underlying formula as SAT.

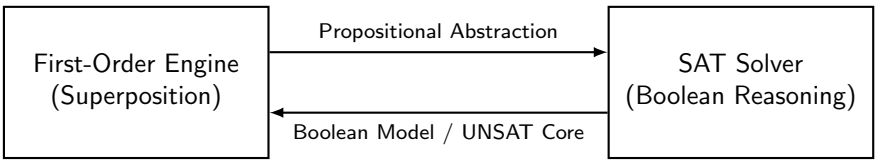


Figure 5.1: Conceptual diagram of the AVATAR architecture adapted from [15]. The workload is split between a standard superposition engine and an embedded SAT solver, exchanging abstractions and models to efficiently determine satisfiability.

# Bibliography


[1] Hajnal Andréka, Johan van Benthem, and István Németi. Modal languages and bounded fragments of predicate logic. *Journal of Philosophical Logic*, 27(3):217–274, 1998.

[2] Leo Bachmair and Harald Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3):217–247, 1994.

[3] Alonzo Church. A note on the Entscheidungsproblem. *Journal of Symbolic Logic*, 1(1):40–41, 1936.

[4] Herbert B. Enderton. *A Mathematical Introduction to Logic*. Academic Press, San Diego, CA, 2 edition, 2001.

[5] Melvin Fitting. *First-Order Logic and Automated Theorem Proving*. Springer, New York, NY, 2 edition, 1996.

[6] Kurt Gödel. Die vollständigkeit der axiome des logischen funktionenkalküls. *Monatshefte für Mathematik und Physik*, 37(1):349–360, 1930.

[7] Erich Grädel. On the restraining power of guards. *Journal of Symbolic Logic*, 64(4):1719–1742, 1999.

[8] Erich Grädel, Phokion G. Kolaitis, and Moshe Y. Vardi. On the decision problem for two-variable first-order logic. *Bulletin of Symbolic Logic*, 3(1):53–69, 1997.

[9] Michael Huth and Mark Ryan. *Logic in Computer Science: Modelling and Reasoning about Systems*. Cambridge University Press, Cambridge, UK, 2 edition, 2004.

[10] Laura Kovács and Andrei Voronkov. First-order theorem proving and Vampire. In *Proceedings of CAV*, volume 8044 of *LNCS*, pages 1–35, 2013.

[11] Ian Pratt-Hartmann, Wieslaw Szwast, and Lidia Tendera. Quine's fluted fragment revisited. In *Proceedings of LICS*, 2018.

[12] Willard V. Quine. Algebraic logic and predicate functors. *The Ways of Paradox and Other Essays*, 1969.

[13] Balder ten Cate and Luc Segoufin. Unary negation. *Logical Methods in Computer Science*, 9(3), 2013.

[14] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936.

[15] Andrei Voronkov. AVATAR: The architecture for first-order theorem provers. In *Computer Aided Verification (CAV)*, volume 8559 of *Lecture Notes in Computer Science*, pages 696–710. Springer, 2014.

[16] Alfred North Whitehead and Bertrand Russell. *Principia Mathematica*, volume 1–3. Cambridge University Press, Cambridge, 1910–1913.